



Google Apps Script란?

구글에서 제공하는 **무료 자동화 툴**입니다.

- Google Sheets, Docs, Gmail 등 다양한 Google 서비스의 작업을 자동화할 수 있습니다.
- **자바스크립트(JavaScript)** 기반이며, 별도 설치 없이 구글 시트에 내장되어 있습니다.
- 매크로보다 더 **강력하게** 사용 가능합니다.



Google Sheets에서 Apps Script 열기

- 1 구글 시트에서 파일을 엽니다.
- 2 상단 메뉴에서: **확장 프로그램** → **Apps Script** 클릭
- 3 새 에디터 탭이 열리면 기본 코드 제공:

```
function myFunction() {  
  
}
```

- 이곳에서 코드 작성을 시작합니다.



첫 번째 스크립트 - "Hello, world!" 쓰기

```
function writeHello() {  
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();  
  sheet.getRange("A1").setValue("Hello, world!");  
}
```

실행 방법:

- 1  실행 버튼 클릭
- 2 권한 승인
- 3 A1 셀에 'Hello, world!' 표시됨

이 스크립트는 현재 활성화된 스프레드시트의 A1 셀에 텍스트를 기록합니다.



주요 개념

개념	의미	예시
SpreadsheetApp	구글 스프레드시트 접근 서비스	<code>SpreadsheetApp.getActiveSpreadsheet()</code>
getActiveSpreadsheet()	현재 활성화된 스프레드시트 파일	<pre>const ss = SpreadsheetApp.getActiveSpreadsheet()</pre>
getActiveSheet()	현재 활성화된 시트(탭)	<code>const sheet = ss.getActiveSheet()</code>
getRange("A1")	특정 셀이나 범위 선택	<code>sheet.getRange("A1")</code>
setValue()	셀에 값 쓰기	<code>range.setValue("Hello")</code>
getValue()	셀에서 값 읽기	<code>const value = range.getValue()</code>

이러한 메서드들을 활용하여 스프레드시트 데이터를 읽고 쓰는 자동화 스크립트를 작성할 수 있습니다.



예제 2 - 두 셀 합계

```
function sumCells() {  
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();  
  const a = sheet.getRange("A1").getValue();  
  const b = sheet.getRange("A2").getValue();  
  sheet.getRange("A3").setValue(a + b);  
}
```

코드 설명:

- 1 A1, A2 셀의 값을 읽어옵니다
- 2 두 값을 더합니다
- 3 결과값을 A3 셀에 기록합니다

작동 방식:



이 예제를 응용하여 복잡한 계산이나 여러 셀 데이터를 처리할 수 있습니다.



예제 3 - 오늘 날짜 표시

```
function fillDates() {  
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();  
  const lastRow = sheet.getLastRow();  
  sheet.getRange(lastRow + 1, 1).setValue(new Date());  
}
```

기능 설명:

- 1 `getLastRow()`로 시트의 마지막 데이터 행을 파악합니다.
- 2 다음 빈 행(A열)에 **오늘 날짜**를 추가합니다.

이 스크립트는 데이터 로그를 기록하거나 타임스탬프를 추가할 때 유용합니다. 트리거와 함께 사용하면 매일 자동으로 날짜를 기록할 수 있습니다.



`new Date()` 는 JavaScript에서 현재 날짜와 시간을 생성합니다.



Google Spreadsheet 엔티티

엔티티	의미	예시
SpreadsheetApp	전체 스프레드시트 앱	<code>SpreadsheetApp.getActiveSpreadsheet()</code>
스프레드시트	파일 단위	<code>SpreadsheetApp.openById()</code>
시트(Sheet)	개별 탭	<code>ss.getSheetByName("Data")</code>
범위(Range)	셀 또는 셀 블록	<code>sheet.getRange("B2")</code>
셀(Cell)	단일 셀	A1, B3 등
값(Value)	셀 내부의 콘텐츠	텍스트, 숫자, 수식 등

이러한 엔티티들은 계층 구조를 가지며, 스프레드시트 데이터에 체계적으로 접근할 수 있게 해줍니다.



셀 접근 단계별 설명

```
const spreadsheet = SpreadsheetApp.getActiveSpreadsheet(); // 전체 파일
const sheet = spreadsheet.getSheetByName("Sheet1"); // 탭 선택
const range = sheet.getRange("A1"); // 셀 선택
const value = range.getValue(); // 값 읽기
```

1 스프레드시트 접근

전체 구글 시트 파일에 접근

2 시트 선택

이름으로 특정 시트(탭) 선택

3 셀 선택

원하는 셀이나 범위 지정

4 값 읽기

선택한 셀의 내용을 변수에 저장

각 단계는 순차적으로 진행되며, 이전 단계의 객체를 활용하여 다음 단계를 실행합니다. 이 기본 패턴은 모든 Apps Script 코드에서 유용하게 사용됩니다.



다양한 Range 예제

1

```
sheet.getRange("B2:B5");
```

4행 범위 선택 (B2부터 B5까지)

2

```
sheet.getRange(2, 2);
```

B2 셀 선택 (행, 열 번호 사용)

3

```
sheet.getRange(2, 2, 3, 1);
```

B2부터 B4까지 (3행 1열) 선택

getRange() 메서드는 다양한 방식으로 셀 범위를 선택할 수 있습니다:

- A1 표기법: "A1:C3"
- 행/열 좌표: getRange(행, 열)
- 범위 지정: getRange(시작행, 시작열, 행수, 열수)

12
34

읽기 & 쓰기 실습

```
const value = sheet.getRange("C1").getValue(); // 읽기
sheet.getRange("C2").setValue("Hello");        // 쓰기
```

값 읽기

getValue() 메서드는 선택한 셀이나 범위에서 값을 읽어옵니다.

- 숫자는 숫자형으로 반환
- 날짜는 Date 객체로 반환
- 수식은 계산된 결과 반환

값 쓰기

setValue() 메서드는 선택한 셀이나 범위에 값을 기록합니다.

- 문자열, 숫자, 날짜 입력 가능
- "=SUM(A1:A5)"와 같은 수식 문자열도 가능
- 셀 서식은 변경하지 않음

💡 대량의 데이터를 처리할 때는 getValues()와 setValues()를 사용하는 것이 더 효율적입니다.

↻ 효율적인 데이터 처리 (베스트 프랙티스)

```
const data = sheet.getRange(2, 1, 100, 3).getValues();
```



100행 x 3열 일괄 읽기

한 번의 호출로 다수의 셀을 읽어 API 호출 수를 최소화합니다.



2차원 배열 반환

data[행][열] 형태로 접근이 가능합니다. (예: data[0][1]은 2행 2열 값)



메모리 내 빠른 처리

개별 셀 접근 대신 메모리에서 데이터를 빠르게 처리할 수 있습니다.

 **핵심:** 여러 셀에 접근할 때는 항상 `getValues()`와 `setValues()`를 사용하세요. 반복문에서 개별 셀 접근보다 수십 배 빠릅니다.



모범 사례

모범 사례	이유
함수명을 명확하게 작성	함수의 목적을 쉽게 식별하고 코드 유지보수 시 이해하기 쉽습니다.
복사본에서 먼저 테스트	중요한 데이터의 손실을 방지하고 안전하게 테스트할 수 있습니다.
주석(//) 적극 활용	나중에 코드를 디버깅하거나 수정할 때 코드의 의도를 이해하기 쉽습니다.
Logger.log() 사용	테스트 중 변수 값이나 실행 흐름을 확인하여 디버깅을 용이하게 합니다.
반복문 대신 일괄 처리	getValues()/setValues()를 사용하여 한 번에 데이터를 읽고 쓰면 실행 속도가 크게 향상됩니다.

위 모범 사례를 따르면 코드의 가독성, 유지보수성, 성능이 크게 향상됩니다.



트리거: 자동 실행

- 1 Apps Script 에디터에서  **아이콘** 클릭 → 트리거 추가

트리거 유형:



수정 시(On edit)

스프레드시트가 수정될 때마다 스크립트 실행



폼 제출 시(On form submit)

새 설문지 응답이 제출될 때 스크립트 실행



시간 기반(Time-driven)

매시, 매일, 매주 등 정해진 시간에 자동 실행

- 2 실행할 **함수 선택** 및 권한 설정

트리거를 활용하면 수동 실행 없이도 데이터 자동 업데이트, 보고서 생성, 알림 발송 등이 가능합니다.

실전 활용 사례

활용 예시	설명
 GA4/BigQuery 데이터 가져오기	API를 통해 자동으로 리포트 데이터를 수집하고 시트에 기록합니다. 주기적인 데이터 수집을 자동화하여 분석 작업의 효율성을 높입니다.
 월간 보고서 자동화	매월 보고서 템플릿을 자동으로 복사하고, 새로운 데이터를 자동 입력하여 정기 보고서 작성 시간을 단축시킵니다.
 알림 이메일 전송	시트의 특정 조건(예: 데드라인, 재고 부족 등)이 충족되면 자동으로 관련자에게 알림 이메일을 발송합니다.
 데이터 정리	데이터 형식 표준화, 중복 제거, 오류 검사 등을 자동화하여 데이터 품질을 향상시키고 정리 작업에 소요되는 시간을 절약합니다.

이러한 자동화 활용 사례들은 단순 반복 작업을 줄이고, 데이터 분석과 의사결정에 더 많은 시간을 투자할 수 있게 해줍니다.



요약 - 오늘 배운 내용

목표	엔티티/메서드
스프레드시트 제어	<code>SpreadsheetApp</code>
시트 선택	<code>getSheetByName()</code> / <code>getActiveSheet()</code>
셀 선택	<code>getRange()</code>
값 읽기	<code>getValue()</code>
값 쓰기	<code>setValue()</code>
효율적 데이터 처리	<code>getValues()</code> / <code>setValues()</code>

이 기본 메서드들을 활용하여 Google Sheets 데이터를 자동화하고 프로그래밍 방식으로 관리할 수 있습니다.

✨ 다음 단계: Google Apps Script로 자동화 리포트 만들기



더 배우고 싶다면?



Google Apps Script 공식 문서

구글에서 제공하는 공식 가이드와 레퍼런스 문서로 모든 기능을 자세히 알아보세요.



구글 개발자 유튜브

실용적인 튜토리얼과 최신 업데이트 소식을 영상으로 확인하세요.



다양한 예제 따라하기

커뮤니티에서 공유되는 실제 사용 사례와 코드를 분석하고 응용해보세요.



실습 경험이 가장 좋은 공부법입니다!

다음 단계 추천 과정

- Google Sheets API 활용하기
- GA4 데이터 자동 가져오기
- 이메일 자동화 리포트 만들기